

Competitive Programming

Section 62: Codeforces Course Ladder

Competitive Programming Course

2026-07-16

Table of contents

1	Why a 50-problem single-source ladder after a broad course	3
2	How to work one ladder problem	4
2.1	The 25-minute rule	4
3	Gate 1: mechanical correctness	5
4	Gate 2: graph and DP modeling	6
5	Gate 3: state compression and structures	7
6	Gate 4: reductions and advanced optimization	8
7	Gates 5–7: expert to red	9
8	A disciplined upsolve protocol	10
9	Offline workflow	11
10	Exercises	12
10.1	A. Gate log	12
10.2	B. Model comparison	12
10.3	C. Compression audit	12
11	Submission checklist	13

1 Why a 50-problem single-source ladder after a broad course

The course has taught tools by idea. Codeforces now supplies a deliberately ordered environment in which the statement is unfamiliar but the underlying move should be recognizable. This is not a rating checklist: a 1400 can expose an unlearned invariant more effectively than a rushed 2200.

Use [problem_sheet.pdf](#) offline. It gives a concise original restatement, the input/output contract, and a visible official URL for every task. The linked Codeforces page is authoritative if a detail differs. Do not open [editorial.pdf](#) until you have recorded an attempt.

2 How to work one ladder problem

Give every attempt an evidence trail:

```
first model | rejected model or counterexample | final invariant | test that changed my mind
```

For example, on 279B, writing “try every start and extend” is not enough. The useful record is:

```
positive reading times -> a valid window stays valid after removing from left  
invariant -> [left, right] is feasible after the while-loop  
test -> one book longer than the full budget
```

That record explains why two pointers are valid and makes the same pattern available on a different story.

2.1 The 25-minute rule

For Gates 1 and 2, work alone for 25 focused minutes. If there is no model, write the exact missing observation and read only the corresponding editorial block. For Gates 3 and 4, use 40–60 minutes because modeling, not typing, is usually the work. Do not replace an attempt with passive editorial reading.

After reading an editorial, close it and reproduce four things from memory:

1. the state or invariant;
2. the transition or greedy choice;
3. why a tempting alternative is wrong;
4. the asymptotic bound in the statement’s variables.

3 Gate 1: mechanical correctness

Task	Course bridge	What must become automatic
71A	Sections 1–2	input loop, length boundary, exact output
4C	Section 3	a map whose value has a precise meaning
363B	Sections 4–5	fixed-window sum and first-index tie handling
279B	Section 6	a monotone two-pointer invariant
158B	Sections 9–10	greedy groups with a proof by forced partners
706B	Section 8	<code>upper_bound</code> / <code>bisect_right</code> as a counting query

Do not race this gate. A wrong answer here should create a named regression test: empty-looking edge cases are where conventions become habits.

4 Gate 2: graph and DP modeling

Task	Course bridge	Recognition question
520B	Section 15	what are the states and unit-cost moves?
455A	Section 22	what future conflict is summarized by one index?
300C	Sections 29–32	which digit sums can be constructed and counted?
166B	Section 54	how does a convex fan locate one polygon vertex?
20C	Section 19	are all edge weights nonnegative, and do I need a parent?
161D	Section 26	how many nodes at every bounded depth lie below me?

For 161D, use the bounded k from the statement to keep a depth-count DP at each tree node. The ladder is ordered by dependency, not merely by rating.

5 Gate 3: state compression and structures

Task	Course bridge	Main danger
580D	Section 27	the last item is part of the DP state
1245D	Section 18	a virtual node turns stations into MST edges
271D	Sections 43 and 55	equality needs collision-aware substring identity
61E	Section 36	strict inequalities and duplicate values in Fenwick queries
86D	Sections 40 and 42	keep the current range statistic updated both ways

At this gate, write a state table before coding. It should say what changes when one item, vertex, character, or array endpoint is added and removed.

6 Gate 4: reductions and advanced optimization

Task	Course bridge	Reduction to state explicitly
498C	Section 45	prime-factor units, odd positions, even positions
319C	Section 49	each earlier DP state is a line queried at $a[i]$

The hard part is often proving the reduction, not selecting a library template. On 498C, show why a unit of a prime cannot be used twice. On 319C, derive the line slope and intercept from the recurrence before choosing a hull.

7 Gates 5–7: expert to red

Tasks 20–50 deliberately stop looking like one-technique exercises. They are still ordered, but the order measures the amount of model compression, proof, and engineering required rather than a single online rating number.

Gate	Tasks	What changes in your attempt routine
5	20–27	derive a representation that turns many operations into logarithmic work
6	28–40	prove the reduction before choosing the advanced structure
7	41–50	derive the optimized recurrence or transformation from a slow oracle

For Gates 5 and 6, spend at least 45 minutes before reading an editorial. For Gate 7, first write a slow exact model and state precisely which monotonicity, decomposition, or algebraic identity must justify the optimization. A correct template with an unproved reduction is not a solved problem.

8 A disciplined upsolve protocol

After a failed attempt, identify the earliest false sentence in your work:

Failure	Repair exercise
wrong tool	list three statement signals and the competing tools
correct tool, wrong state	write two histories that reach the same old state but need different futures
proof gap	state the exchange, induction, or cut argument in three lines
implementation bug	preserve the smallest failing test and label the convention
complexity miss	count states and transitions independently

Then solve a nearby task only after you can explain the repaired invariant without looking. The editorial gives a route to understanding; it should not become a substitute for retrieving the route yourself.

9 Offline workflow

The sheet is intentionally concise enough to print. Before a travel day or an offline session:

1. render this section with `make pdf`;
2. save `problem_sheet.pdf` and `editorial.pdf` locally;
3. hide the editorial until the attempt time ends;
4. once connected, submit through the visible Codeforces URL and compare any judge failure with the sheet's stated contract.

The URLs remain useful as references in a printed PDF even when they cannot be clicked. They also avoid pretending that a course copy replaces the official judge.

10 Exercises

10.1 A. Gate log

Solve the first four tasks without editorials. For each, write the invariant and one smallest hostile test. Advance only when all four descriptions are independent of the story wording.

10.2 B. Model comparison

For 20C, 1245D, and 498C, draw the graph used by the solution. Mark what each node and edge means, then explain why every valid original choice maps to one graph object and back.

10.3 C. Compression audit

For 455A, 580D, 86D, and 319C, name precisely what information is kept and what information is deliberately discarded. Give a counterexample showing why discarding one more field would fail.

11 Submission checklist

- ☐ I solved the offline restatement, not a remembered answer.
- ☐ My notes contain a state/invariant and a disproof of one tempting approach.
- ☐ The complexity counts the actual constrained quantity.
- ☐ I preserved one test that would catch my original mistake.
- ☐ I read the official linked statement before submitting.